Informazioni cronologiche	Username (group members)	Train Accuracy	Validation Accuracy	Test Accuracy	Method
05/12/2025 18.55.45	Fantin Anthony, Picello Davide	99.91	97.28	96.94	Loaded a pretrained model: inception-v4. Fine tuned the entire network for 10 epochs using two parallelized T4 optimizer: AdamW criterion: LabelSmoothingCrossEntropy Utilized a scheduler to decrease the learning rate to guarantee improving at each epoch, specifically CosineAn batch size: 128, doubled having 2 GPUs Data augmented with pytorch: horizontal flips, equalization. Images resized to match the ones used to pretrain the network.
03/12/2025 18.49.16	Riccardo Niccolò Agosti	98.71%	96.42%	96.24 %	Fine-tuning of a pretrained ResNet50d on CIFAR-10 using the adamW optimizer with a learning rate of 1e-4 and
02/12/2025 12.35.52	D'Ovidio Giacomo, Furno Francesco	97.36	96.58	96.04	Transfer learning using ResNet50. First, we trained only the classifier with a learning rate of 5e-4, and then we
05/12/2025 0.44.38	Dario Liotta - Gabriel Amorosetti	99.98%	95.51%	95.59%	We trained a custom ResNet-like CNN where each stage uses downsampling in the first block and identity sho
01/12/2025 17.51.05	D'Ovidio Giacomo, Furno Francesco	96.83	95.08	94.93	Transfer learning using EfficientNet-B0, the base model of the EfficientNet family. First, we trained only the cla
02/12/2025 17.31.34	Giulio Morandini, Marco Venanzi	98.97	99.51	94.34	ResNet18 and Data Augmentation Key Parameters: Learning rate: 1e-3 Epochs: 30 Optimizer: SGD with Nesterov momentum Regularization: Weight decay = 1e-4 We used ResNet18 pre-trained on ImageNet and adapted it to CIFAR-10's 32×32 images. We applied data augmentation (horizontal flips and random cropping).
04/12/2025 11.04.21	Giovanni Chessa	97.77	94.57	94.34	I built a Residual Net with 4,623,178 parameters, that outperformed completely the CNNs, I also tried fine tunir I used AdamW as the optimizer, with little weight decay (5e-4) and learning rate = 1e-3. I used a scheduler with patience 7 I did a lot of data augmentation, which prevented me from overfitting, and was really the only way to go (I tried -transforms.ColorJitter(brightness=0.1, contrast=0.1, saturation=0.1), -transforms.RandomRotation(10), -transforms.RandomCrop(32, padding=4), -transforms.RandomHorizontalFlip(p=0.5), -transforms.RandomErasing(p=0.5, scale=(0.02, 0.33), ratio=(0.3, 3.3)) I ran for 50 epochs.
05/12/2025 17.13.32	Giuseppe Castellana	99.47%	95.06%	94.32%	Custom CNN modeled on a classical ResNet-18 architecture. Used 128 batch size and different augmentation
04/12/2025 19.44.57	Milan Riccardo	98.50%	94.10%	93.85%	The third strategy I used was a ResNet to solve the degradation problem, employing skip connections to prese
30/11/2025 21.19.56	Mahsa Ekrahi	99.15%	99.62%	93.74 %	I used pretrained resnet18. adjusted the number of out_channels in first conv layer to 64, set the first pooling la
02/12/2025 18.10.14	Arturo Bevilacqua	99.91	93.06	93.61	I defined and trained a Residual Network with the following architecture: • Stem: 1x [Conv + BatchNorm + ReLU] (3x32x32 -> 64x32x32) • 3xStages, each one composed of 3xResidual Blocks, first one with stride=2 and doubling channels (64x32x3 • Average Pooling + FC layer Data Augmentation: RandomCrop, RandomHorizontalFlip, RandomErasing (note: I removed ColorJitter and Ra Optimizer: Adam with lr=0.001 Weight initialization: kaiming normal Training on gpu with early stopping and "custom learning rate decay" (i.e. if no validation accuracy improveme
26/11/2025 16.29.54	Vladimir Ungureanu	98.43%	92.98%	93.49%	Number of Layers: 3 blocks each with 2 convolutions and 1 pooling Number of Neuron per layer: 65,536; 65,536;16,384; 32,768; 32,768; 8,192; 16,384; 16,384; 256 Activation functions: ReLU Weight initialisation: Kaiming Dropout rate: 0.5 Learning rate:0.1 Number of epochs:200 Optimiser: Nesterov Batch size: 256 Data augmentation: random crop, random horizontal flip, normalisation, random erasing
04/12/2025 21.27.40	Elia Carta	99.91%	99.00%	93.32 %	I started by applying data augmentation to the CIFAR-10 training set, specifically using random horizontal flips Then, I built a custom VGG-style network with residual blocks. The model consists of 4 stages (convolutional l For the training, I used the Cross Entropy loss function and the SGD optimizer (lr=0.1, momentum=0.9, weight_ The model was trained for 60 epochs.
04/12/2025 15.39.12	Sobhan Hosseini	90.7	N/A (not used)	93.21	Architecture: A custom convolutional model based on a from-scratch implementation of ResNet-34. Training Setup: The training pipeline included standard spatial augmentation (random crop, horizontal flip) together with stron These augmentations deliberately increase input variability, which reduces overfitting and leads to a lower trai
04/12/2025 13.47.46	Sobhan Hosseini	98.60	N/A (not used)	93.03	Architecture: A custom CNN based on a from-scratch implementation of ResNet-34, modified with a lightweight non-linear c The model replaces the standard fully connected classifier with a small MLP-style module to improve feature u Training Techniques: Standard data augmentation (random crop, horizontal flip), SGD with momentum, weight decay, cosine-anneal Trained for 25 epochs from scratch on CIFAR-10.
04/12/2025 16.17.20	Davide Sut	98.00%	98.89%	92.92%	I've used a modified pretrained version of RestNet18. To fit the 32x32 size of CIFAR-10 I've removed the maxpooling layers, modified the first convolutional layer to b For the optimizer I've decided to use AdamW and for the loss function the cross entropy loss. Some of the parameters are: learning rate=1e-3, N_EPOCHS=24, weight_decay=1e-2. Random cropping ad horizontal flips data augmentation.
05/12/2025 22.58.16	zahraa saleh	94.35%	91.29%	92.13%	I implemented a CNN with six convolutional layers. Each layer had 3×3 convolutions, batch normalization, ReLU
05/12/2025 22.28.30	Alessandro Murru	99.98%	92.75%	92.13 %	I modified the train and model_training functions to accelerate the simulation training process and to impleme

Informazioni cronologiche	Username (group members)	Train Accuracy	Validation Accuracy	Test Accuracy	Method
05/12/2025 21.35.34	Edoardo Florian (216602)	98.51	99.03	92.06	Used a pre-trained model, densenet121, freezed all the features layers besides the last dense block of the mod Resized the images to 224x224 to match densenet121 architecture. Used Imagenet mean and std for normalization, since densenet121 is trained on imagenet Used SGD optimizer with lr=0.005, momentum=0.9, weight decay=0.0005. Trained for 20 epochs
05/12/2025 18.13.26	Edoardo Florian (2166022)	93.84	96.04	91.60	I implemented a cnn using pytorch I tried to use a resnet architecture based on residual blocks, a residual block contains a skip connection and is y = conv + batch norm + relu + conv + batch norm x = skip-connection relu(y + x) Then the overall model architecture is the following -stem: convolution + batch norm + relu -residual block 1 -residual block 2,3,4: like 1 but use stride 2 in one of the convolution for downsampling -global average pooling -one fully connected layer with 10 outputs. Data augmentation -RandomHorizontalFlip(p=0.5), -RandomCrop(32, padding=4) I changed the Adam optimizer with the SGD one with lr= 0.05, momentum= 0.9 and weight decay=0.0005 I also added the CosineAnnealingLR scheduler. Trained for 100 epochs
29/11/2025 16.20.21	zahraa saleh	88.55%	89.51%	91.36%	I did a CNN with three convolutional blocks, each having two 3×3 convolutions, batch normalization, ReLU, drop
04/12/2025 19.40.29	Milan Riccardo	96.20%	92.15%	91.36%	In my second strategy I used a VGG-inspired architecture utilizing stacks of small 3x3 filters for deep hierarchi
03/12/2025 16.03.17	D'Ovidio Giacomo, Furno Francesc	92.5	91.6	90.7	Custom CNN with four convolutional blocks (Conv–BN–ReLU) and fractional max pooling, which gradually red
02/12/2025 22.37.08	Jovan Cakic	95.73%	91.29%	90.62%	Transform pipeline applies data augmentation like color jitter, random horizontal flip and random perspective, a I used CNN with VGG styled architecture with nine convolutional layers grouped into three blocks, each followe The training is done with batch size 256, using SGD with momentum, weight decay and ReduceLROnPlateau s
30/11/2025 15.18.04	Walter Troiani	98.93	99.57	90.35	Performing full-finetuning into a pretrained 19-layer VGG variant with the following hyperparameters and some LEARNING_RATE = 1e-4 WEIGHT_DECAY=1e-5 BATCH_SIZE=64 OPTIMIZER = "ADAM"
05/12/2025 20.36.43	Hooman Sabzi	89.65	90.43	90.31	I have implemented an MLP classifier that begins with five convolutional blocks, each containing a Conv2d lay
30/11/2025 12.32.03	Matteo Garato	98.80	99.51	90.22	ConvNeXt pretrained model was imported and an FClayer with 10 output nodes was added at the end of the m Optimizer: AdamW with learning_rate=1e-4 and weight_decay=1e-5 Data augmentation used to better generalize the training: RandomRotation(15), RandomHorizontalFlip(p=0.6), RandomVerticalFlip() The batch size was reduced to 32. The model was trained for 30 epochs
05/12/2025 9.04.52	Baro Giada	91,27%	90.86%	90.18%	I used a batch_size = 128. I modify the model function, using tree hidden layers with always a ReLu activation f
26/11/2025 13.12.18	Walter Troiani	93.45%	97.03%	89.12	Performing full-finetuning into a pretrained 19-layer VGG variant with the following hyperparameters: LEARNING_RATE = 1e-4 BATCH_SIZE=64
05/12/2025 18.11.17	Fusari, Micino	97.52%	98.69%	88.71%	Custom CNN based on a classical VGG16 architecture. We added: - data augmentation techniques: - on training data RandomCrop(32, padding=4),RandomHorizontalFlip() e Resize(64) - on test data Resize(64) - early stopping mechanism with patience 5 Parameters modified: - optimizer: SGD with lr=0.005, momentum=0.9, weight_decay=5e-4 - n_epochs = 50
05/12/2025 22.39.27	Alessandro Murru	95.66%	90.05%	88.71 %	I optimized the weight initialization function for efficiency. A four-block CNN was designed where each block c On the last block i used the Adaptive Avarage Pool. Classification is performed by a final fully connected module. The model was trained using the AdamW optim
04/12/2025 16.07.16	Andrade, Brule	95.4	98.32	88.59	Transfer learning using RESNET50 archicteture with pretrained weights, and then performing fine-tuning. Mode
05/12/2025 20.59.00	Marangon, Mietto	92.86%	89.18%	88.43 %	Pre-processing: normalization using mean and std. Data augmentation: RandomHorizontalFlip, RandomCrop(size=(32,32), padding=4), ColorJitter(brightness=0.2 Pre-Trained Architecture: model: efficientnet_b7(weights=EfficientNet_B7_Weights.IMAGENET1K_V1) all layer have been freezed, the only trained layer was the last fully connected Batch size = 128 number of epochs = 50 optimizer = Adam(model.parameters(), lr=1e-3) Loss= CrossEntropyLoss()
05/12/2025 23.00.07	Alessandro Murru	96.17%	89.21%	87.96 %	I refined the weight initialization method to improve efficiency. I developed a Convolutional Neural Network (CNN) architecture, inspired by the VGGNet framework. The model was optimized using the AdamW algorithm in conjunction with the OneCycleLR scheduler.

Informazioni cronologiche	Username (group members)	Train Accuracy	Validation Accuracy	Test Accuracy	Method
02/12/2025 17.41.59	Elio Shyti, Max Horn	88.44%	88.49%	87.87%	Applied data augmentation with RandomRotation, RandomHorizontalFlip and Normalize. Loss = Cross entropy loss Optimizer = Adam with lr = 1e-3 N_EPOCHS = 25 Model architecture: [(Conv -> BatchNorm -> ReLU)×2 -> MaxPool -> Dropout] × 3 -> Flatten -> (FC -> BatchNorm-> ReLU -> Dropout) 1st block: 64 filters 4x4, padding = 1, maxpool = 2x2, dropout = 0.2 2nd block: 128 filters 4x4, padding = 1, maxpool = 2x2, dropout = 0.25 3rd block: 128 filters 4x4, padding = 1, maxpool = 2x2, dropout = 0.35 The last dropout after the linear layer is equal to 0.5
03/12/2025 13.39.10	Lazzari Tommaso	99.995	89.33%	87.67%	Data augmentation + fine tuning of resnet18 with a modified first convolutional layer, that allows to take in inp
05/12/2025 12.32.49	Vullnet Vogli	94.35	86.87	86.69	Implemented Network in Network CNN with global average pooling. AdamW optimizer, ReduceOnPlateau scheduler and kaiming/xavier weigth init.
26/11/2025 20.36.35	Frances Ferrabee	93.36	87.78	86.68	Did CNN ensemble with 3 pretrained models: resnet18,resnet34 and mobilenet. Added data augmentation such as random crop,and random horizontal flip. Added a scheduler CosineAnnealingWarmRestarts and used Adam as the optimizer with learning rate 3e-4, an
05/12/2025 22.37.09	Arduino	93.3%	94.53%	86.54%	To train the VGG16 model, pretrained on ImageNet, the Adam optimizer was configured with a minimal learnin
05/12/2025 15.34.17	Marangon, Mietto	94.08%	87.77%	86.42%	Pre-processing: normalization using mean and std. Data augmentation: RandomHorizontalFlip, RandomCrop(size=(32,32), padding=4), ColorJitter(brightness=0.2 Network Architecture: - The convolution part is organized in 4 layers. Layer 1: convolution with spatial dimensions preserved, no pooling. Layers 2-4: conv(doubling channels) ->conv(same channels) -> MaxPool2d(kernel_size=2, stride=2) -> skip co - global average pooling (output dimension 256). - linear layer (output dimension 10). activation function=ReLU batchnormalization2d before applying ReLU Weight Initialization: He Normal (Kaiming) batch size = 128 N_epochs=100 optimizer = Adam(model.parameters(), lr=1e-4, weight_decay=1e-4) Loss= CrossEntropyLoss()
05/12/2025 22.49.47	Fusari, Micino	93.40%	96.84%	86.25%	Custom CNN based on ResNet18 architecture. We added: - data augmentation techniques: - on training data RandomCrop(32, padding=4),RandomHorizontalFlip() e Resize(64) - on test data Resize(64) - early stopping mechanism with patience 5 Parameters modified: - optimizer: SGD with lr=0.005, momentum=0.9, weight_decay=5e-4 - n_epochs = 60
30/11/2025 10.49.57	Garato Matteo	84.51	91.33	85.30	CNN from scratch: the architecture used is similar to VGG16, with Dropout2d(p=0.3) added after each convolutional layer, furtherm Optimizer: Adam with learning_rate=1e-4 and weight_decay=1e-5 Data augmentation used to better generalize the training: RandomRotation(15), RandomHorizontalFlip(p=0.6), RandomVerticalFlip() The batch size was reduced to 64. The model was trained for 40 epochs
03/12/2025 15.23.52	Marco Augusto Carturan, Andrea C	82.53%	81.17%	81.17%	In our second trial, we implement a CNN with 3 convolutional layer (in each of them we use ReLU activation fu
02/12/2025 20.37.47	Tharen Candi	80.05%	80.75%	80.14%	Transfer Learning from pytorch mobilenetv2 IMAGE NET. lr04 FC only 20 epochs then lr06 with FC + LAST CO

Informazioni cronologiche	Username (group members)	Train Accuracy	Validation Accuracy	Test Accuracy	Method
03/12/2025 23.50.12	Ana Sofia Salcido, Pankhuri Chatur	72.17	76.90	77.25	We did a CNN based on a VGG Network with the next structure: Convolution (3, 32, 3, 1) BatchNorm2d(32) ReLU Convolution (3, 32, 3, 1) BatchNorm2d(32) ReLU MaxPool(2) Convolution(32, 64, 3, 1) BatchNorm2d(64) ReLU Convolution(64, 64, 3, 1) BatchNorm2d(32) ReLU MaxPool(2) Flatten() Linear(2048, 256) ReLU() Dropout(0.4) Linear(256, 10). We use BatchNorm2d to stabilize and speed up training by normalizing the activations of eac For training, we use a batch size of 256, cross entropy loss, and the Adam optimizer with learning rate 3e−4 an We apply the following data augmentation techniques: HorizontalFlip, RandomCrop, RandomRotation, GaussianBlur, and ColorJitter. Weights are initialized using Kaiming normal initialization, which is appropriate for networks using ReLU activa
05/12/2025 22.36.50	Marangon, Mietto	% model 2 = 72.91% m%	model 2 = 74.75% m	75.11%	Ensamble Method: The ensamble is made up by three models with a majority voting strategy. Common features: - Pre-processing: normalization using mean and std. - Data augmentation: RandomHorizontalFlip, RandomCrop(size=(32,32), padding=4), ColorJitter(brightness=0. - optimizer = Adam(model.parameters(), lr=1e-3) - Loss= CrossEntropyLoss() - batch size = 64 First Model: (Focus on more general features) activation function=ReLU N_epochs=100 Convolution part: - Layer 1: – convolution with 4 output channels, kernel dimention 5, padding = 2 – relu – maxpool with kernel dimension = 2 - Layer 2: – convolution with 8 output channels, kernel dimention 5 – relu – maxpool with kernel dimension = 2 Fully connected layers: Alternation of linear 3 layers and 2 relu Second Model: (Inspired from LeNet) activation function=ELU N_epochs = 75 Convolution part: - Layer 1: – convolution with 6 output channels, kernel dimention 5, padding = 2 – elu – maxpool with kernel dimension = 2 - Layer 2: – convolution with 16 output channels, kernel dimention 5 – elu – maxpool with kernel dimension = 2 Fully connected layers: Alternation of linear 3 layers and 2 elu Third Model: (Focus on more abstract features) activation function=ReLU N_epochs=50 Convolution part: - Layer 1: – convolution with 8 output channels, kernel dimention 3, padding = 1 – relu – maxpool with kernel dimension = 2 - Layer 2: – convolution with 16 output channels, kernel dimention 3, padding = 1 – relu – maxpool with kernel dimension = 2 - Layer 3: – convolution with 32 output channels, kernel dimention 3, padding = 1 – relu – maxpool with kernel dimension = 2 Fully connected layers: Alternation of linear 3 layers and 2 relu
05/12/2025 18.47.47	Ponchio Mattia	89.78%	93.71%	72.13%	2nd Submission, Custom Network: CNN part: Conv2d(3, 64, kernel_size=8, padding="same"), nn.ReLU(), Conv2d(64 , 128 , kernel_size=4 , stride = 2, padding = 1), nn.ReLU(), MaxPool2d(4, stride = 1, padding = 0), Conv2d(128 , 256 , kernel_size = 2 , stride = 2, padding = 1), nn.ReLU(), Conv2d(256 , 512 , kernel_size = 2, stride = 2, padding = 1), nn.ReLU(), MaxPool2d(2, stride = 2, padding = 0) Fully connected: (flattened array has size 2048) 4 Linear layers separated by ReLU and Dropout with out_features, respectively, 4096, 4096, 1024, 10. 60 epochs, Adam optimizer, 1e-5 lr batch size=64
27/11/2025 13.58.11	Arturo Bevilacqua	85.64	66.34	66.92	Ensembling of MLPs: added RandomCrop, ColorJitter, RandomErasing and RandomFlip to the data augmentat

Informazioni cronologiche	Username (group members)	Train Accuracy	Validation Accuracy	Test Accuracy	Method
01/12/2025 14.06.19	Lorenzo Pitti	77.5	64.8	66.49	Ensembling of MLPs. Used Batch=128. Used simple augmentation (RandomCrop and RandomHorizontalFlip). Used Pre-Activation BatchNorm and GELU activations. Regularized with Dropout (=0.1 for deep and =0.2 for wi Final preds aggregated via soft voting.
05/12/2025 16.09.27	Ponchio Mattia	67.24%	68.28%	64.76%	1st submission. I recreated the vgg16 architecture (30 layer CNN and 6 layer Fully-Connected) and trained it from scratch, addi Finally, I experimented with the addition of dropout in the convolutional part via torchvision.ops.DropBlock2d(
25/11/2025 16.29.40	Vladimir Ungureanu	61.54%	62.08%	63.52%	Number of Layers: 7(with the input and output layers) Number of neurons per layer: 3072, 2048, 2048, 1024, 512, 256, 10 Activation functions: All ReLu Weights initialisation : Kaiming Dropout Rate: 0.3 Learning rate: Cosine decay Number of epochs:100 Optimizer: Adam Batch size: 256 Data Augmentation:random shifts, random orientations, normalisation
01/12/2025 18.07.40	Zaupa Riccardo	64.12%	62.45%	62.03%	Fully connected MLP architecture with the structure: 3072, 1024, 512, 256, 10. Each layer uses batch normaliz
24/11/2025 23.09.02	Franciszek Sulima	63.24	62.32	61.64	I did MLP with skip connection: 3072 -> 788 -> 788 -> 788 -> 10 (skip connection was summing first and secon
26/11/2025 10.59.54	Frances Ferrabee	68.51%	60.67%	60.41 %	I did 5 layers excluding the input and output layer with one skip connection 3000-> 1500 -> 1500 -> 750 -> 300 ->150 I changed from RELU to LeakyRelu to fix the potential dying RELU problem To help with regularization I added dropout at each layer ranging from 0.5 at the first layer to 0.05 at the last la I used an adam optimizer with learning rate 5e-4 and rate decay 1e-4 along with ReduceLROnPlateau schedule I ran this for 150 Epochs
04/12/2025 19.36.15	Milan Riccardo	68.40%	61.10%	60.25%	In my first approach I used an ensembling technique to reduce model variance and improve predictive robustn
04/12/2025 21.25.21	Alberto Fabris	67.23	61.03	59.45	MLP architecture with batch size = 128 combined with RandomCropping and padding set to 4. Furthermore i a
02/12/2025 17.32.43	Marco Augusto Carturan, Andrea C	44.68%	58.89%	58.89%	We implement a deep MLP for CIFAR-10, with input size of 3072 and four hidden layers of 2048, 1024, 512, and
26/11/2025 14.23.04	Alessandro Del Gallo	68.45%	57.79%	56.32%	MLP funnel and Batch Normalization 3072 to 1024 to 512 to 256 to and output 10 Learning rate= 1e-3 Early stopping, batch size 256
25/11/2025 17.32.02	Garato Matteo	62.98%	57.14%	55.59%	the architecture used is: [input size: 3072] -> [2048] -> ReLU -> Dropout(0.3) -> [1024] -> ReLU -> Dropout(0.2) -> [512] -> ReLU -> Dropout Optimizer: Adam with learning_rate=1e-4 and weight_decay=1e-5 Dropout is used to prevent that some neurons become predominant Data augmentation used to better generalize the training: RandomRotation(15),RandomHorizontalFlip(p=0.6),R The batch size was reduced to 128. The model was trained for 100 epochs
18/11/2025 20.55.21	Baseline	54.40%	54.41%	54.37%	Default approach with dropout, Adam optimizer, 5-layer MLP, and weight initialization
01/12/2025 19.02.49	marinalimabraga	1.00	0.9328	0.9242	Used 18 layers, Conv1 of 64 filters, 512 feature channels and a final FC layer of 10 neurons. The activation function used was ReLU in all convolutional blocks. For the weights initialization, it was used a pretrained ImageNet weights. There was no dropout rate used in the chosen architecture. The learning rate was 0.01 and the number of epochs 40. About the optimizer, it was used SGD with a momentum of 0.9 and a weight_decay of 1e-4. The batch size was 128 because of the GPU I had. For Data augmentation, it was used a padding of 4 and a horizontalflip of p= 0.5. It was used different training techniques, label smoothing of 0.1, it was also used the learning rate scheduler. It was used a pre-trained network called ResNet18.
05/12/2025 20.43.05	Alfeo	0.90	0.90	0.90	I used a VGG-Style architecture, utilizing five convolutional blocks. Each block uses 3x3 convolutions with batc
05/12/2025 20.19.38	Alfeo	0.91	0.89	0.89	I used a transfer-learning strategy with a reduced VGG architecture adapted for CIFAR-10. I used a lighter VGG
04/12/2025 17.38.03	Giulia Agazzi	92	82	84	ResNet (8 residual blocks); Bach size= 256; dropout rate=0.3 in all residual blocks; learning rate=1e-3; Weight d
05/12/2025 20.17.11	Meritxell Pena	67,23	74,35	74,93	Data & preprocessing – Applied data augmentation: RandomRotation(15°), RandomHorizontalFlip(p=0.6), ColorJitter, GaussianBlur, a Loss & optimizer – Loss: CrossEntropyLoss – Optimizer: Adam, lr = 1e-4, weight_decay = 1e-5 – Number of epochs: 25 Model architecture (V4) – A VGG-like CNN composed of: [(Conv → BatchNorm → LeakyReLU) × 2 → MaxPool → Dropout2d] × 3 → Flatten → Linear(2048 → 256) → LeakyReLU → Dropout(0.5) → Linear(256 → 10) – Block details: Block 1: 32 filters (3×3), padding=1, MaxPool 2×2, Dropout2d=0.3 Block 2: 64 filters (3×3), padding=1, MaxPool 2×2, Dropout2d=0.3 Block 3: 128 filters (3×3), padding=1, MaxPool 2×2, Dropout2d=0.3
01/12/2025 18.14.15	Zaupa Riccardo	96.5%	92.8%	91%	Custom CNN was trained for 25 epochs (batch size 64) using the Adam optimizer (lr = 0.0005) and early stopp

Informazioni cronologiche	Username (group members)	Train Accuracy	Validation Accuracy	Test Accuracy	Method
04/12/2025 10.53.02	Giovanni Chessa	90%	88%	88%	I built a CNN with 5 convolutional layers (each one of them with a ReLU and maxpool, except for the last one w In total 44 millions of parameters I used AdamW as the optimizer, with little weight decay (5e-4) and learning rate = 1e-3. I also used a scheduler with patience 5. I also did data augmentation, which was the key in every kind of network I tried for this competition. More spec -RandomCrop(32, padding=4), -RandomHorizontalFlip(p=0.5)
24/11/2025 22.13.49	Walter Troiani	80%	59%	60%	Made the MLP more "residual" by adding Skip Connections and LayerNorm, and small dropouts of 20% (Initial LEARNING_RATE=1e-4 WEIGHT_DECAY=1e-5 BATCH_SIZE=64 N_EPOCHS = 100
05/12/2025 20.34.25	Arduino	56.72%	56.24%	56%	The model is a deep Multi Layer Perceptron, composed of five fully connected layers. Each block uses the ReLU activation function, dropout (0.5 → 0.4 → 0.3) to reduce overfitting, and LayerNorm The model is trained for 35 epochs, using the AdamW optimizer, a moderate learning rate of 0.0003, and weigh
05/12/2025 22.20.15	Alice Mameli	52323%	43,22%	37,57%	Hyperparameter gridsearch